



Test Target: Synnefo.academy

Vulnerability Assessment and Penetration Testing Report

Shijimol_S

Novavi Student

March 11, 2026

Contents

0.1 Executive Summary.....	3
0.2 Key Findings	3
0.2.1 Severity.....	3
0.2.2 Vulnerability Severity Distribution	4
0.2.3 Issue Categorization.....	4
1 Vulnerability Details	5
1.1 Medium Findings.....	5
1.1.1 Clickjacking.....	5
1.2 Low Findings.....	7
1.2.1 CORS Misconfiguration.....	7
1.3 Informational Findings.....	10
1.3.1 Public File Metadata Exposure via API.....	10
1.3.2 Technology Stack Disclosure	12
1.3.3 Public Static Asset Enumeration	12
2 Conclusion	14

0.1 Executive Summary

In March 2026, Novavi engaged Shijimol S, a cybersecurity student at Novavi, to perform a comprehensive Vulnerability Assessment and Penetration Testing (VAPT) of the Synnefo.academy web application. It is an official website of Synnefo Academy, an IT training institute that provides professional courses in areas like cybersecurity, DevOps, cloud computing, networking, Python, and full-stack development. The objective of the engagement was to perform a holistic security review of the Synnefo.academy website.

- Manual penetration testing and code-assisted reconnaissance of web endpoints.
- Automated scanning with industry-standard tools such as Nessus. Development of proof-of-concept (PoC) demonstrations for vulnerabilities. Throughout the engagement, periodic consultations were held with the Synnefo.academy development team to discuss testing methodology, interim findings, and remediation strategies.

0.2 Key Findings

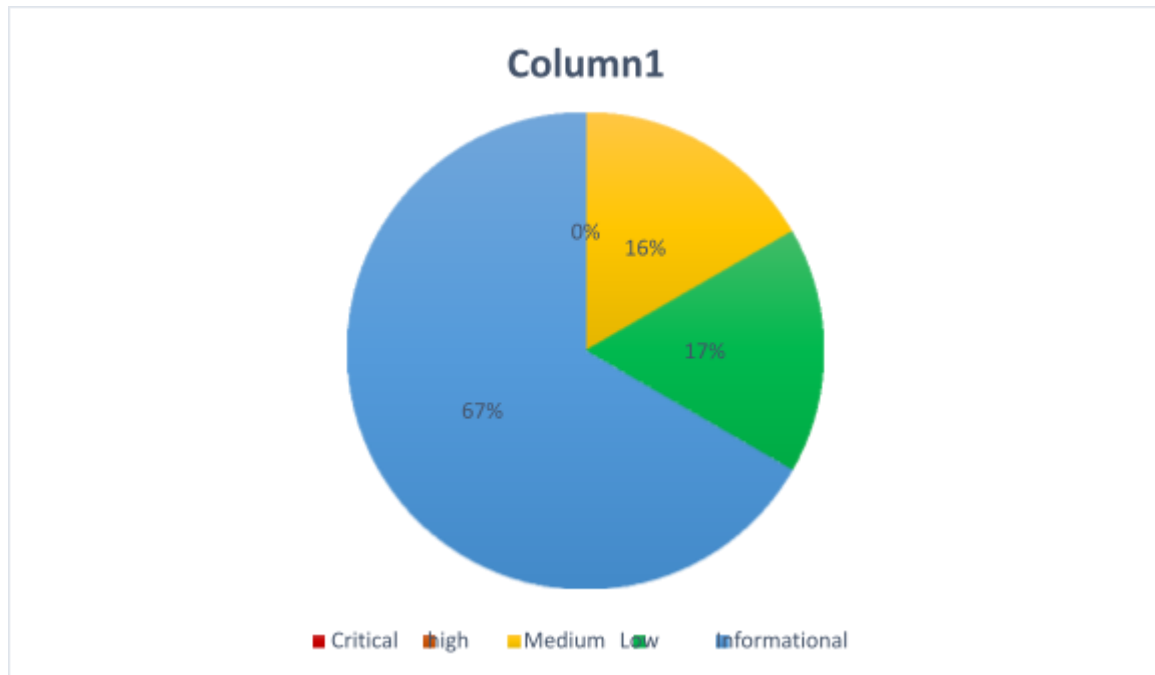
The overall assessment of synnefo.academy indicates a **potential CORS misconfiguration**, which may allow unauthorized cross-origin requests if origin restrictions are not properly enforced, potentially exposing API endpoints to requests from malicious domains.

0.2.1 Severity

Severity	# of Findings
Critical	0
High	0
Medium	1
Low	2
Informational	3

0.2.2 Vulnerability Severity Distribution

Vulnerability Severity Distribution



0.2.3 Issue categorization

Issue Category	# of Findings
Clickjacking	1
CORS Misconfiguration	1
Public File Enumeration via API	1
Technology Stack Disclosure	1
Public Static Asset Enumeration	1

Table 2: Issue categorization

Chapter 1

Vulnerability Details

1.1 Medium Findings

1.1.1 Clickjacking

Severity:  Medium

Description

The synnefo.academy application pages can potentially be loaded inside an external iframe, which may allow attackers to trick users into clicking hidden elements.

Discovery Method

- The clickjacking vulnerability was identified by analyzing the HTTP response headers of the application using **Burp Suite**.
- The application response was checked for the presence of security headers such as X-Frame-Options and Content-Security-Policy frame-ancestors.
- Additionally, the **Clickjacking Tester browser extension** was used to verify whether the webpage could be loaded inside an external iframe.

Impact Analysis

Clickjacking allows attackers to trick users into clicking hidden elements on a webpage by embedding the legitimate website inside a malicious iframe.

An attacker could overlay invisible buttons or frames over legitimate page elements and manipulate users into performing unintended actions such as:

- Clicking sensitive buttons
- Submitting forms
- Performing actions without their knowledge

This could potentially lead to unauthorized actions being performed on behalf of the user.

Proof of Concept

The following HTML code demonstrates how the vulnerable page can be embedded inside an iframe:

```
<html>
<head>
<title>Clickjacking PoC</title>
</head>
<body>
<h2>Clickjacking Test</h2>
<iframe src="https://synnefo.academy"
width="800"
height="600">

</iframe>
</body>
</html>
```

Steps to reproduce:

- 1 Create a file named clickjack.html.
- 2 Open the file in a browser.
- 3 The **synnefo.academy** page loads inside the iframe, the application is vulnerable to clickjacking.

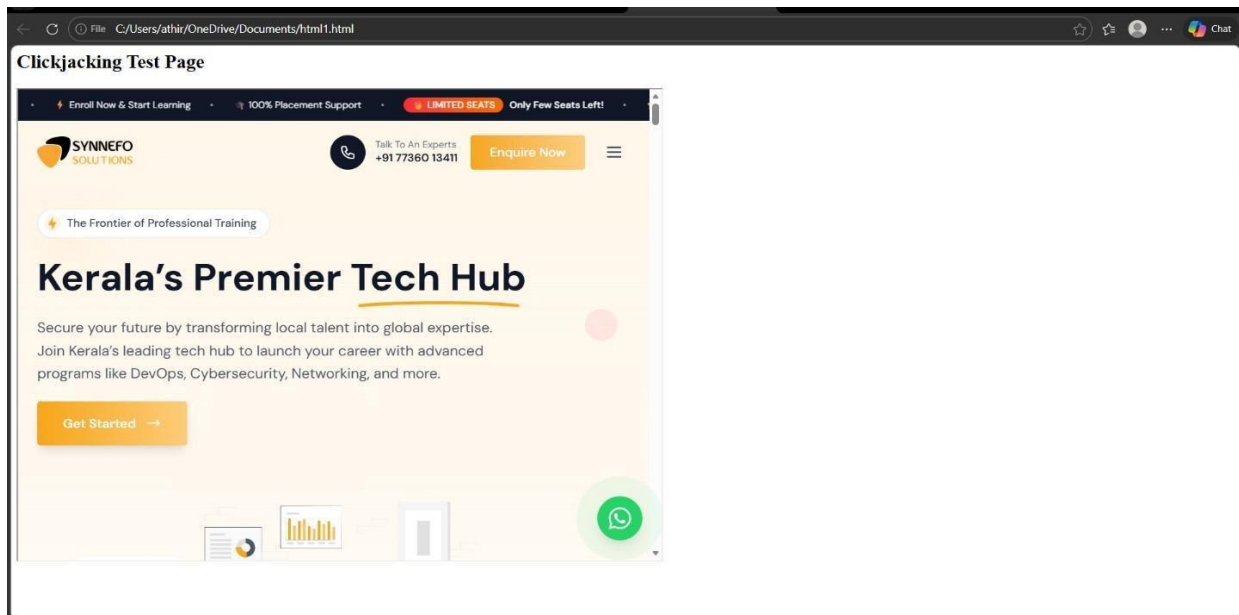


Figure 1.1: Screenshot showing the website loads inside the iframe.

Mitigation

- Implement frame protection
headers: X-Frame-Options:
SAMEORIGIN
- Additionally, implement a Content Security Policy: Content-Security-Policy:
frame-ancestors 'self'

1.2 Low Findings

1.2.1 CORS Misconfiguration

Severity: **Low**

Description

The API response contains CORS headers that allow cross-origin requests.

Discovery Method

- The **Cross-Origin Resource Sharing (CORS)** configuration was tested using **Burp Suite** by modifying the Origin header in HTTP requests.
- The server responses were analyzed to check whether the application allows requests from arbitrary external origins.
- Testing showed that the server responded to requests containing external origins, indicating a potential **CORS misconfiguration**.

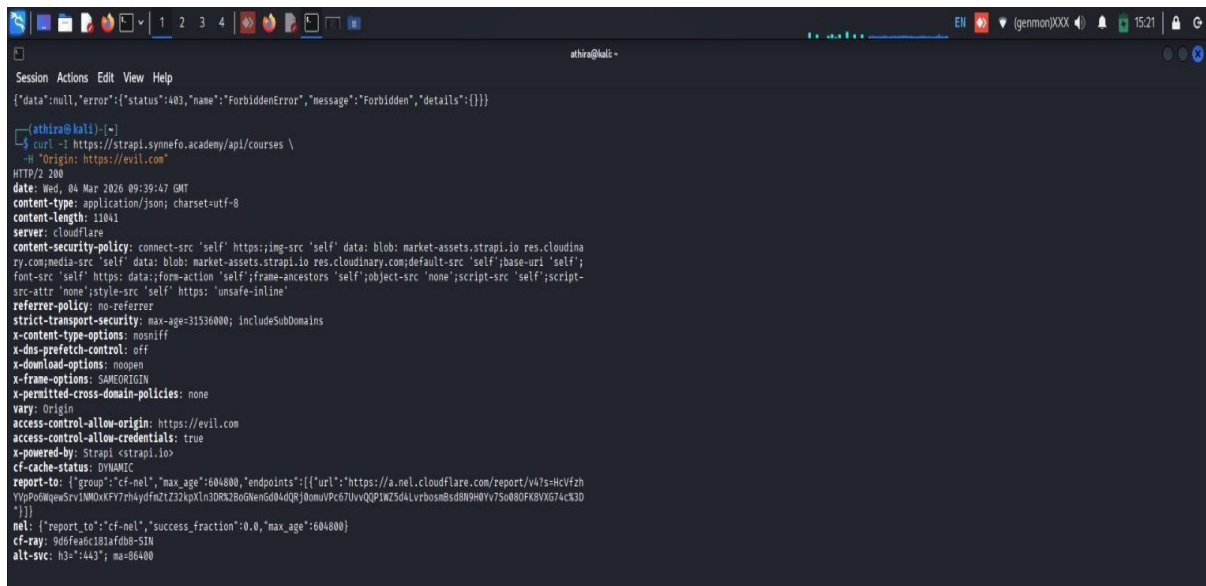
Impact Analysis

- Improper CORS configuration may allow malicious websites to send authenticated requests to the application from a victim's browser.
- This could lead to unauthorized access to sensitive information or execution of unintended actions if authentication tokens or cookies are included in the request.

Proof of Concept

1. The following command was used to test the CORS configuration:

```
$ curl -I https://strapi.synnefo.academy/api \
-H "Origin: https://evil.com"
```

```
athira@kali:~$ curl -i https://strapi.synnefo.academy/api/courses \
-H 'Origin: https://evil.com'
HTTP/2 200
date: Wed, 04 Mar 2026 09:39:47 GMT
content-type: application/json; charset=utf-8
content-length: 11041
server: cloudflare
content-security-policy: connect-src 'self' https://img-src 'self' data: blob: market-assets.strapi.io res.cloudinary.com/media-src 'self' data: blob: market-assets.strapi.io res.cloudinary.com/default-src 'self' base-url 'self' font-src 'self' https://data: form-action 'self' frame-ancestors 'self' object-src 'none' script-src 'self' script-src-attr 'none' style-src 'self' https://unsafe-inline
referrer-policy: no-referrer
strict-transport-security: max-age=31536000; includeSubDomains
x-content-type-options: nosniff
x-dns-prefetch-control: off
x-download-options: noopen
x-frame-options: SAMEORIGIN
x-permitted-cross-domain-policies: none
vary: Origin
access-control-allow-origin: https://evil.com
access-control-allow-credentials: true
x-powered-by: Strapi <strapi.io>
cf-cache-status: DYNAMIC
report-to: {"group": "cf-nel", "max_age": 604800, "endpoints": [{"url": "https://a.nel.cloudflare.com/report/v4?ts=HcVfzhYVpPoWqewSrvNM0xKF7rh4ydfnZLZ32kpLn3DRX2BoGnGd04dQ8J0omUPC67UvVQQP1NZ5d4Lvrbsm8sd8N9H0V750080FK8YXG74c3D"}]}}
nel: {"report_to": "cf-nel", "success_fraction": 0.0, "max_age": 604800}
cf-ray: 906feab181afdb0-SIN
alt-svc: h3=443; ma=60400
```

Figure 1.2: Screenshot indicates that the server allows requests from untrusted origins.

2. During testing, the application was analyzed using **Burp Suite Repeater** by modifying the Origin header in the HTTP request.

- The request was sent with a **malicious origin domain (https://hacker.com)** to check if the server validates the origin before allowing cross-origin requests.

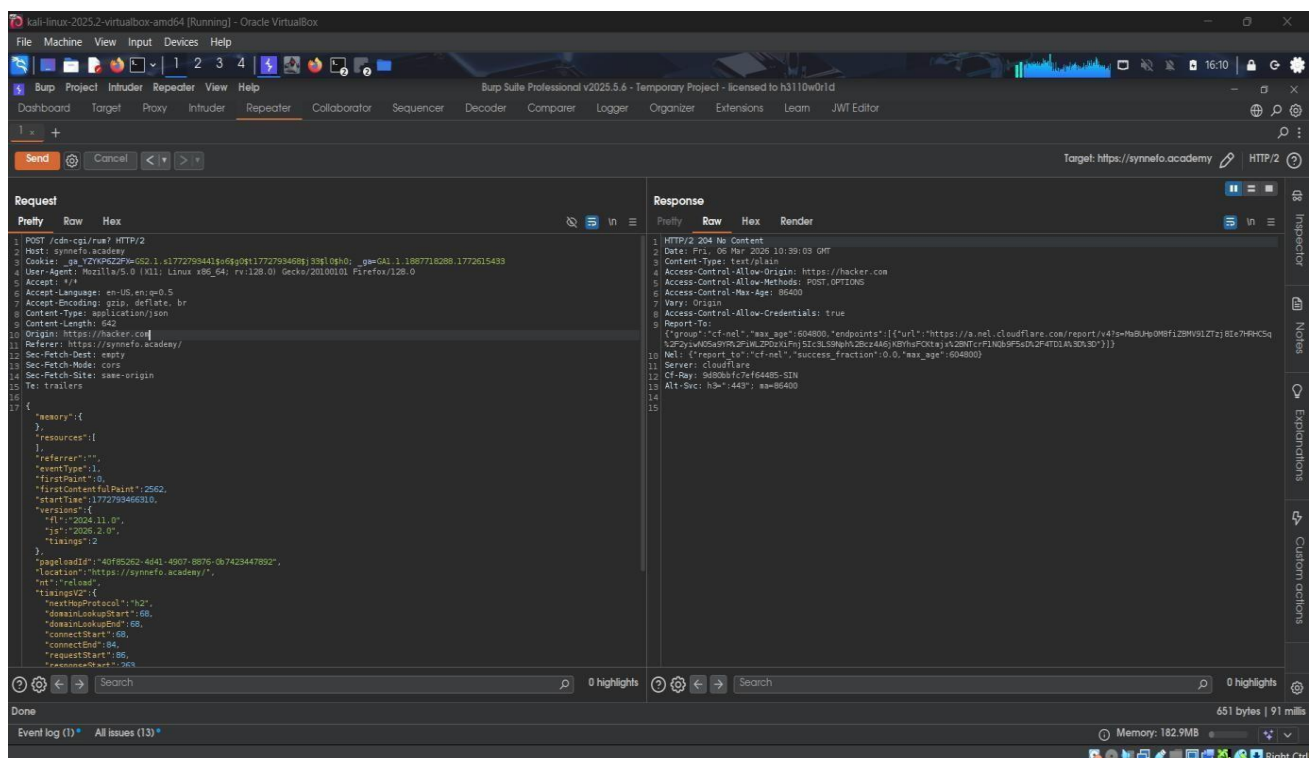


Figure 1.2: Screenshot indicates that the server reflects the attacker-controlled origin (<https://hacker.com>).

Mitigation

- Restrict allowed origins to trusted domains only.
- Avoid using wildcard (*) origins in sensitive APIs.
- Disable Access-Control-Allow-Credentials unless absolutely required.
- Implement strict server-side access control validation.

1.3 Informational Findings

1.3.1 Public File Metadata Exposure via API

Severity

Informational

Description

- The endpoint `/api/upload/files` in **synnefo.academy** allows unauthenticated users to retrieve metadata of uploaded files. The API returns details such as file name, URL, type, and size without requiring authentication, allowing attackers to enumerate publicly stored assets.

Discovery Method

- Manual API testing using **curl** was performed on the endpoint <https://strapi.synnefo.academy/api/upload/files>.
- The response returned a list of uploaded files and metadata without requiring authentication, confirming that the API allows public enumeration of uploaded file information.

Impact Analysis

- Attackers can enumerate uploaded files.
- Exposure of file metadata and storage paths.
- Potential disclosure of sensitive files if misconfigured in the future.

Proof of Concept

The vulnerability can be reproduced by directly accessing the public API endpoint that lists uploaded files.

- 1 **Send Request to API Endpoint:** An unauthenticated HTTP request was sent to the Strapi file API endpoint:

\$ **curl** "<https://strapi.synnefo.academy/api/upload/files>"

- 2 **Server Response:** The server responds with a JSON object containing metadata of uploaded files.

```
{
  "id": 1,
  "name": "course-banner.png",
  "hash": "course_banner_123",
  "ext": ".png",
  "mime": "image/png",
  "size": 245.32,
  "url": "/uploads/course_banner.png",
  "createdAt": "2026-02-05T10:21:15.000Z"
}
```

- 3 **Direct File Access:** Using the returned URL, the file can be accessed directly from the browser.

Eg: https://strapi.synnefo.academy/uploads/course_banner.png

This confirms that uploaded file metadata and file URLs are accessible without authentication.

Security Impact: This may lead to:

- Information disclosure
- Enumeration of internal file structure.
- Potential exposure of sensitive files if misconfigured in the future

Mitigation:

- Restrict access to the endpoint via authentication
- Limit metadata exposure for public users
- Apply proper access control rules in Strapi API permissions

1.3.2 Technology Stack Disclosure

Severity: Informational

Discovery Method:

- Application fingerprinting and response analysis revealed that the backend CMS used by the application is Strapi.
- This information was identified through HTML responses and script resources loaded from the admin interface.

Proof of Concept (PoC)

Accessing the admin endpoint reveals Strapi resources:

<https://strapi.synnefo.academy/admin>

Observed Impact: Technology disclosure allows attackers to identify the framework used by the application. This information can be used to search for known vulnerabilities affecting specific versions of the framework.

Mitigation:

- Minimize technology disclosure where possible.
- Remove unnecessary framework identifiers from responses.
- Regularly update the CMS to the latest secure version.

1.3.3 Public Static Asset Enumeration

Severity: Informational

Description: The application allows public access to certain uploaded assets stored under the /uploads/ directory. Some files are

accessible directly via URL if the correct file name is known. This may allow attackers to enumerate publicly stored assets.

Proof of Concept (PoC)

- Example request:
https://strapi.synnefo.academy/uploads/mern_b2bfd24763.png
- HTTP Request:
Curl https://strapi.synnefo.academy/uploads/mern_b2bfd24763.png

Observed Impact

- Attackers may discover publicly stored files
- Could expose unintended assets if misconfigured in the future

Mitigation

- Restrict directory listing
- Store sensitive files outside the public directory
- Use signed URLs for file access when required

Chapter 2

Conclusion

- The Vulnerability Assessment and Penetration Testing (VAPT) conducted on Synnefo Academy and its associated API services hosted on Strapi provided insight into the current security posture of the platform.
- The assessment identified several medium, low, and informational security findings related to API configuration, security headers, and access controls. While no critical vulnerabilities were discovered during the engagement, the identified issues may assist attackers in reconnaissance or enable secondary attacks if combined with other vulnerabilities.

Key Findings:

- **Clickjacking Vulnerability:**

The application does not implement proper frame protection headers such as X-Frame-Options or Content-Security-Policy. As a result, attackers could embed the website within a malicious iframe to trick users into performing unintended actions.

- **CORS Misconfiguration:**

Improper Cross-Origin Resource Sharing configuration may allow unauthorized domains to interact with the API. This could potentially enable cross-origin data access or facilitate malicious web-based attacks.

- **Public File Enumeration via API:**

The endpoint `/api/upload/files` exposes metadata of uploaded files without authentication. Although the files appear to be public assets used by the website, this behavior may allow attackers to enumerate stored resources and map internal file structures.

3. Informational Findings

- Missing or weak security headers
- Server and framework information disclosure
- Public API endpoint enumeration

These issues primarily aid attackers in reconnaissance and attack surface mapping.

4. Proof-of-Concept Validation

All identified vulnerabilities were validated through controlled testing using manual techniques and security testing tools. Proof-of-concept (PoC) demonstrations confirmed the presence of the issues and their potential exploitability.

5. Recommendations

To improve the overall security posture of the platform, the following actions are recommended:

- Implement strict CORS policies allowing only trusted domains.
- Enable anti-clickjacking protections using X-Frame-Options or Content-Security-Policy.
- Restrict access to sensitive API endpoints where possible.
- Implement proper security headers across all responses.
- Conduct regular security assessments and code reviews.
- Integrate secure development practices into the software development lifecycle.

6. Strategic Security Considerations

- Although the identified vulnerabilities are not critical, addressing them will significantly improve the platform's resilience against reconnaissance, web attacks, and potential exploitation chains.

- A proactive approach to security—including continuous monitoring, periodic penetration testing, and adherence to secure coding practices—will help ensure long-term protection of the platform and its users.

In conclusion, Synnefo Academy's platform demonstrates functional stability; however, several configuration-related security issues should be addressed to strengthen its overall defense posture. Timely remediation of the identified findings will reduce the attack surface, enhance application security, and help maintain user trust.